

Patrones de diseño implementados

1. Mediator

Ubicación:

```
backend/src/application/mediator/Mediator.ts
```

El patrón Mediator evita que los controladores conozcan directamente todos los servicios y casos de uso. Cada controlador envía un comando o una consulta al mediador, y el mediador decide qué handler debe ejecutarse.

Ejemplo conceptual:

```
QuotesController -> mediator.send("quotes.create", payload) -> CreateQuoteHandler
```

Ventajas:

- Menos acoplamiento entre controladores y lógica de negocio.
- Casos de uso más fáciles de reemplazar o probar.
- Estructura ordenada para agregar nuevos RF.

2. Strategy

Hay dos usos principales del patrón Strategy.

2.1 Estrategias de cálculo de cotización

Ubicación:

```
backend/src/application/quotes/strategies/pricing
```

Estrategias:

- FixedPackagePricingStrategy : paquetes con precio fijo.
- PerChildPricingStrategy : paquetes que dependen del número de niños/personas.
- CustomPricingStrategy : cotizaciones personalizadas por rubros manuales.

El sistema elige la estrategia según el tipo de paquete y los datos enviados desde el wizard.

2.2 Estrategias de cambio de estado

Ubicación:

```
backend/src/application/quotes/strategies/status
```

Estrategias:

- AcceptQuoteStatusStrategy : bloquea la fecha en agenda.
- ReleaseQuoteStatusStrategy : libera la fecha si se rechaza o vence.
- DraftOrSentStatusStrategy : mantiene la fecha sin bloquearla definitivamente.

Esto permite cumplir RF14 y RF14.1 sin llenar el caso de uso de condicionales desordenados.